МИНИСТЕРСТВО ОБРАЗОВАНИЯ КИРОВСКОЙ ОБЛАСТИ
Кировское областное государственное профессиональное образовательное бюджетное учреждение
«Слободской колледж педагогики и социальных отношений»

ОТЧЕТ

по учебной практике

ПМ11. Разработка, администрирование и защита баз данных

Студент

Баранцев Евгений Дмитриевич

Группа 23П-2

Специальность 09.02.07 Информационные системы и программирование

Руководитель практики от колледжа:

Калинин Арсений Олегович

Руководитель практики от организации:

_____ *Калинин Арсений Олегович*

подпись

УТВЕРЖДАЮ:

Директор

_____

Наименование организации

_____

_____

_____/_____

подпись расшифровка

М. П.

2025-2026 уч. год

Содержание

Оглавление

- Выполнение анализа и предварительной обработки информации (UML2, ERD)
- Выделение объектов и атрибутов в соответствии с заданием(DATA DICTIONARY)
- Проектирование и нормализация БД в полном соответствии с поставленной задачей и применением CASE-средств (схема бд)
- Реализация уровней доступа для различных категорий пользователей
- Создание запросов и отчетов в соответствии с заданием (представления, хранимые процедуры, триггеры)
- Разработка модулей программы.
- Заключение

Приложения

1. Выполнение анализа и предварительной обработки информации (UML2, ERD)

В ходе практики была выполнена предварительная обработка информации по предметной области системы контроля физического доступа к значимому объекту **«ХранительПРО»**. На этом этапе была изучена постановка задачи, определены основные пользователи системы, их роли, действия и сценарии взаимодействия с системой.

Были выделены основные участники системы:

- посетитель;
- сотрудник общего отдела;
- сотрудник охраны;
- сотрудник подразделения;
- система как автоматический исполнитель отдельных функций.

На основе анализа была построена **диаграмма вариантов использования UML2**, отражающая, какие действия доступны каждому пользователю системы. Диаграмма показывает процессы регистрации, оформления заявок, согласования посещений, контроля доступа, фиксации времени входа и выхода, добавления посетителей в черный список и формирования отчетности.

Также была построена **ERD-диаграмма**, отражающая структуру базы данных и связи между сущностями. На диаграмме были показаны основные таблицы: пользователи, роли, сотрудники, подразделения, заявки, посетители, документы, связи заявок с посетителями, журнал посещений, черный список и другие объекты.

В результате этапа анализа были определены:

- основные бизнес-процессы системы;
- необходимые сущности базы данных;
- связи между сущностями;
- будущая структура программных модулей.

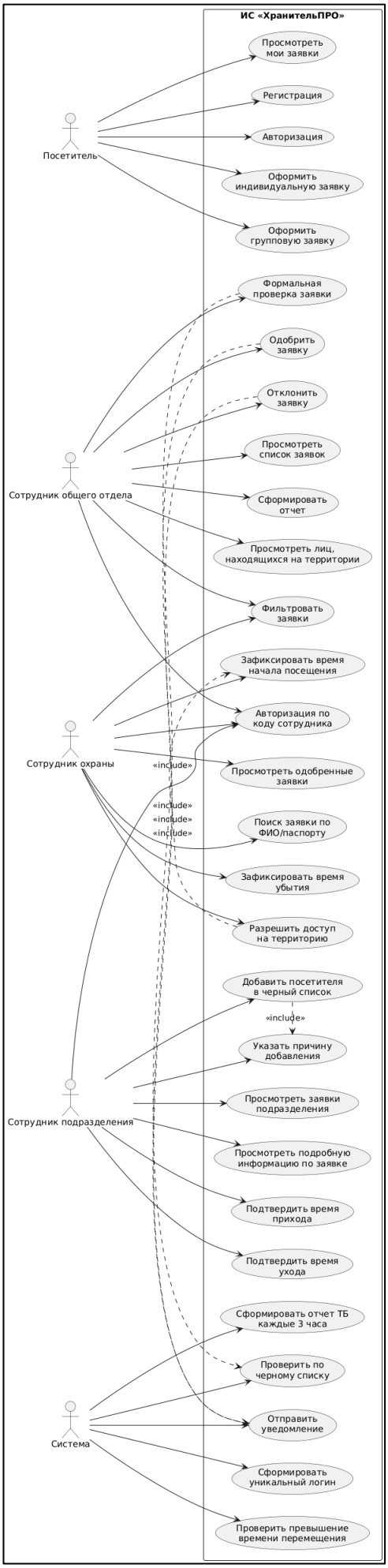

Рисунок 1 – UML-Диаграмма.

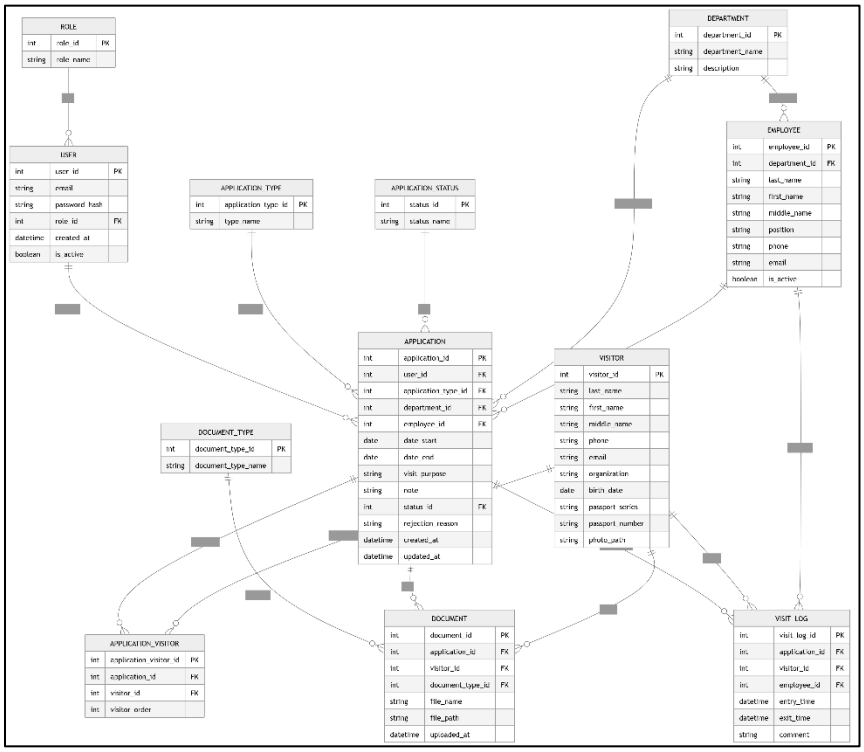

Рисунок 2 – ERD-Диаграмма.

2. Выделение объектов и атрибутов в соответствии с заданием (DATA DICTIONARY)

На данном этапе были выделены основные объекты информационной системы **«ХранительПРО»** и их атрибуты. Для каждого объекта определены поля, типы данных и назначение. Это позволило сформировать словарь данных, который описывает структуру базы данных и показывает, какие сведения используются в системе.

Основными объектами системы являются пользователи, сотрудники, подразделения, посетители, заявки, документы, журнал посещений и черный список. Эти таблицы обеспечивают работу всех модулей программы: клиента посетителя, терминала общего отдела, терминала охраны и терминала сотрудника подразделения.

1. Таблица users

Назначение: хранение учетных записей пользователей системы.

Поле	Тип данных	Описание
user_id	integer	Уникальный идентификатор пользователя
email	varchar	Адрес электронной почты пользователя
password_hash	varchar	Хэш пароля пользователя
login	varchar	Сформированный уникальный логин
role_id	integer	Ссылка на роль пользователя
created_at	timestamp	Дата и время регистрации

is_active	boolean	Признак активности учетной записи

Описание:

Таблица users содержит данные для входа в систему. Пользователь регистрируется по электронной почте и паролю, после чего для него формируется уникальный логин. Поле role_id связывает пользователя с конкретной ролью в системе.

2. Таблица roles

Назначение: хранение ролей пользователей.

Поле	Тип данных	Описание
role_id	integer	Уникальный идентификатор роли
role_name	varchar	Наименование роли

Описание:

Таблица roles определяет категории пользователей системы. На основе ролей реализуется разграничение доступа. В системе используются роли посетителя, сотрудника общего отдела, сотрудника охраны и сотрудника подразделения.

3. Таблица departments

Назначение: хранение сведений о подразделениях организации.

Поле	Тип данных	Описание
department_id	integer	Уникальный идентификатор подразделения
department_name	varchar	Название подразделения
description	text	Описание подразделения
transfer_minutes_limit	integer	Допустимое время перемещения от проходной до подразделения

Описание:

Таблица departments содержит информацию о подразделениях организации. Она используется в заявках, отчетах и терминалах сотрудников. Поле transfer_minutes_limit нужно для контроля превышения времени перемещения посетителей от проходной до подразделения.

4. Таблица employees

Назначение: хранение данных о сотрудниках организации.

Поле	Тип данных	Описание
employee_id	integer	Уникальный идентификатор сотрудника

department_id	integer	Ссылка на подразделение
last_name	varchar	Фамилия
first_name	varchar	Имя
middle_name	varchar	Отчество
position	varchar	Должность
phone	varchar	Контактный телефон
email	varchar	Электронная почта
is_active	boolean	Признак активности сотрудника
employee_code	varchar	Код сотрудника для входа в терминалы

Описание:

Таблица employees используется для хранения сотрудников организации. Через нее реализуется авторизация сотрудников общего отдела, охраны и подразделений по коду. Также таблица используется для назначения принимающего сотрудника в заявке.

5. Таблица visitors

Назначение: хранение данных о посетителях.

Поле	Тип данных	Описание
visitor_id	integer	Уникальный идентификатор посетителя
last_name	varchar	Фамилия
first_name	varchar	Имя
middle_name	varchar	Отчество
phone	varchar	Телефон
email	varchar	Электронная почта
organization	varchar	Организация посетителя
birth_date	date / timestamp	Дата рождения
passport_series	varchar	Серия паспорта
passport_number	varchar	Номер паспорта
photo_path	varchar	Путь к фотографии

Описание:

Таблица visitors хранит персональные данные посетителей, необходимые для оформления и проверки заявок. Один и тот же посетитель может участвовать в нескольких заявках. Эти данные используются при формальной проверке, пропуске на объект и добавлении в черный список.

6. Таблица applications

Назначение: хранение заявок на посещение.

Поле	Тип данных	Описание
application_id	integer	Уникальный идентификатор заявки
user_id	integer	Пользователь, создавший заявку

application_type_id	integer	Тип заявки
department_id	integer	Подразделение, куда оформляется посещение
employee_id	integer	Принимающий сотрудник
date_start	timestamp	Дата начала посещения
date_end	timestamp	Дата окончания посещения
visit_purpose	text	Цель посещения
note	text	Примечание
status_id	integer	Статус заявки
rejection_reason	text	Причина отклонения
created_at	timestamp	Дата создания заявки
updated_at	timestamp	Дата изменения заявки

Описание:

Таблица applications является центральной таблицей базы данных. В ней хранятся все заявки на посещение объекта. Каждая заявка связана с пользователем, подразделением, принимающим сотрудником, типом заявки и статусом. Через эту таблицу проходит весь процесс согласования и контроля посещения.

7. Таблица application_types

Назначение: хранение типов заявок.

Поле	Тип данных	Описание
application_type_id	integer	Уникальный идентификатор типа
type_name	varchar	Название типа заявки

Описание:

Таблица application_types содержит справочник типов заявок. В системе используются, например, индивидуальные и групповые заявки. Это позволяет разделять логику обработки разных видов посещений.

8. Таблица application_statuses

Назначение: хранение статусов заявок.

Поле	Тип данных	Описание
status_id	integer	Уникальный идентификатор статуса
status_name	varchar	Название статуса

Описание:

Таблица application_statuses содержит справочник статусов заявок. Например: «На проверке», «Одобрена», «Отклонена». Использование отдельной таблицы статусов упрощает фильтрацию, отчеты и обработку заявок на разных этапах.

9. Таблица application_visitors

Назначение: связь заявок с посетителями.

Поле	Тип данных	Описание
application_visitor_id	integer	Уникальный идентификатор записи
application_id	integer	Ссылка на заявку
visitor_id	integer	Ссылка на посетителя
visitor_order	integer	Порядковый номер посетителя в заявке

Описание:

Таблица application_visitors реализует связь между заявками и посетителями. Она особенно важна для групповых заявок, где в одной заявке может быть несколько посетителей. Поле visitor_order позволяет сохранить порядок посетителей внутри заявки.

10. Таблица documents

Назначение: хранение прикрепленных документов.

Поле	Тип данных	Описание
document_id	integer	Уникальный идентификатор документа
application_id	integer	Ссылка на заявку
visitor_id	integer	Ссылка на посетителя
document_type_id	integer	Тип документа
file_name	varchar	Имя файла
file_path	varchar	Путь к файлу
uploaded_at	timestamp	Дата загрузки документа

Описание:

Таблица documents содержит информацию о прикрепленных файлах. Документы необходимы для проверки корректности заявки и используются сотрудником общего отдела при формальной проверке. Например, сюда относятся файлы, подтверждающие личность посетителя.

11. Таблица visit_logs

Назначение: журнал фактических посещений.

Поле	Тип данных	Описание
visit_log_id	integer	Уникальный идентификатор записи
application_id	integer	Ссылка на заявку
visitor_id	integer	Ссылка на посетителя
security_employee_id	integer	Сотрудник охраны
department_employee_id	integer	Сотрудник подразделения
entry_time	timestamp	Время входа на территорию
exit_time	timestamp	Время выхода с территории

department_entry_time	timestamp	Время прихода в подразделение
department_exit_time	timestamp	Время ухода из подразделения
violation_message	text	Сообщение о нарушении временного интервала
comment	text	Комментарий

Описание:

Таблица visit_logs хранит фактические данные о посещениях. Она используется охраной и сотрудниками подразделения для фиксации времени входа, прибытия, ухода и выхода. На основе этих данных строятся отчеты о посещениях и списки лиц, находящихся на территории организации.

12. Таблица black_list_entries

Назначение: хранение записей черного списка.

Поле	Тип данных	Описание
black_list_entry_id	integer	Уникальный идентификатор записи
visitor_id	integer	Посетитель, внесенный в черный список
reason	text	Причина добавления
created_at	timestamp	Дата добавления
added_by_employee_id	integer	Сотрудник, добавивший запись

Описание:

Таблица black_list_entries используется для хранения информации о посетителях, которым ограничен доступ на объект. Запись может быть добавлена автоматически системой или вручную сотрудником подразделения. Таблица применяется при формальной проверке заявки и при последующих попытках оформления посещения.

Вывод по разделу

Таким образом, в словаре данных были выделены основные объекты системы и их атрибуты. Каждая таблица выполняет определенную функцию и связана с другими таблицами через внешние ключи. Это обеспечивает целостность данных и поддержку всех бизнес-процессов системы: от регистрации пользователя до контроля посещения, ведения черного списка и формирования отчетов.

3. Проектирование и нормализация БД в полном соответствии с поставленной задачей и применением CASE-средств (схема БД)

На данном этапе была выполнена разработка структуры базы данных с учетом требований предметной области. База данных проектировалась так, чтобы исключить избыточность, дублирование информации и нарушения целостности данных.

При проектировании использовались принципы нормализации. Структура базы данных была приведена к **третьей нормальной форме (3НФ)**. Это позволило:

- исключить повторяющиеся данные;
- разделить сущности по смыслу;
- обеспечить корректные связи между таблицами;
- упростить дальнейшую поддержку и развитие системы.

В базе данных были реализованы:

- первичные ключи;
- внешние ключи;
- ограничения NOT NULL;
- связи один-ко-многим;
- таблицы-связки для реализации отношений многие-ко-многим;
- справочники типов и статусов.

В качестве CASE-средства использовалась визуальная схема структуры БД, отражающая все таблицы и связи между ними. Схема базы данных позволила наглядно представить архитектуру проекта и проверить правильность связей между сущностями.

Итоговая структура БД обеспечивает поддержку всех основных процессов системы:

- регистрации пользователей;
- оформления заявок;
- обработки заявок сотрудниками;
- контроля посещений;
- ведения черного списка;
- формирования отчетов.

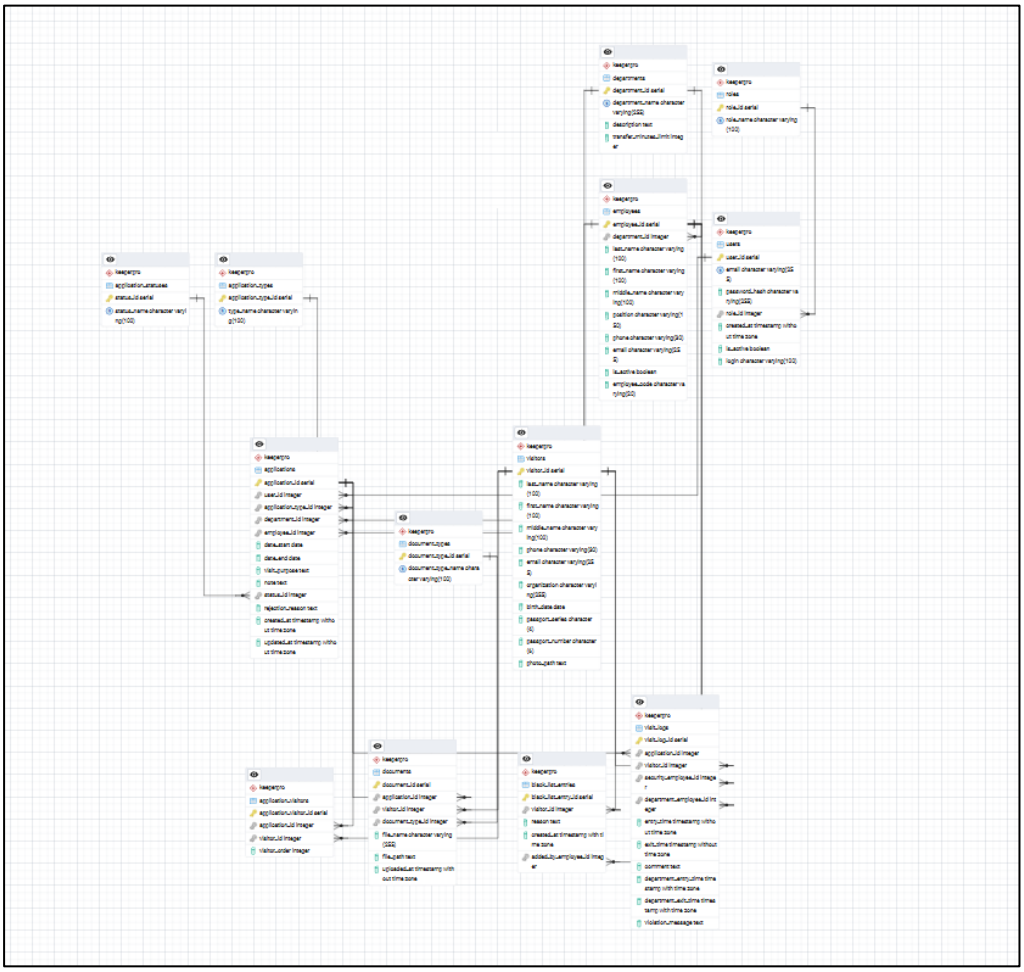

Рисунок 3 – Диаграмма БД.

4. Реализация уровней доступа для различных категорий пользователей

В системе были реализованы уровни доступа для разных категорий пользователей. Это необходимо для разграничения прав и обеспечения безопасности данных.

В проекте были выделены следующие категории пользователей:

- **посетитель**;
- **сотрудник общего отдела**;
- **сотрудник охраны**;
- **сотрудник подразделения**.

Для каждой категории был реализован свой набор функций.

Посетитель

Посетитель может:

- зарегистрироваться в системе;
- войти в систему;
- оформить индивидуальную заявку;

- оформить групповую заявку;
- просматривать свои заявки.

Сотрудник общего отдела

Сотрудник общего отдела может:

- авторизоваться по коду сотрудника;
- просматривать список заявок;
- фильтровать заявки;
- проверять заявку;
- одобрять или отклонять заявку;
- назначать дату и время посещения;
- формировать отчеты;
- просматривать лиц, находящихся на территории.

Сотрудник охраны

Сотрудник охраны может:

- авторизоваться по коду сотрудника;
- просматривать одобренные заявки;
- фильтровать заявки;
- искать заявку по ФИО или паспорту;
- разрешать доступ на территорию;
- фиксировать время начала посещения;
- фиксировать время завершения посещения.

Сотрудник подразделения

Сотрудник подразделения может:

- авторизоваться по коду сотрудника;
- просматривать одобренные заявки своего подразделения;
- подтверждать приход посетителей;
- подтверждать уход посетителей;
- добавлять посетителей в черный список.

Таким образом, каждая роль получила только те функции, которые относятся к ее обязанностям.

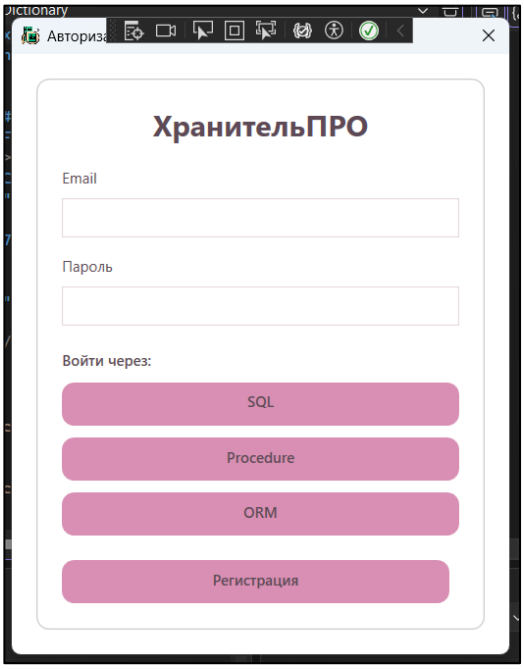

Рисунок 4 – Окно авторизации пользователя.

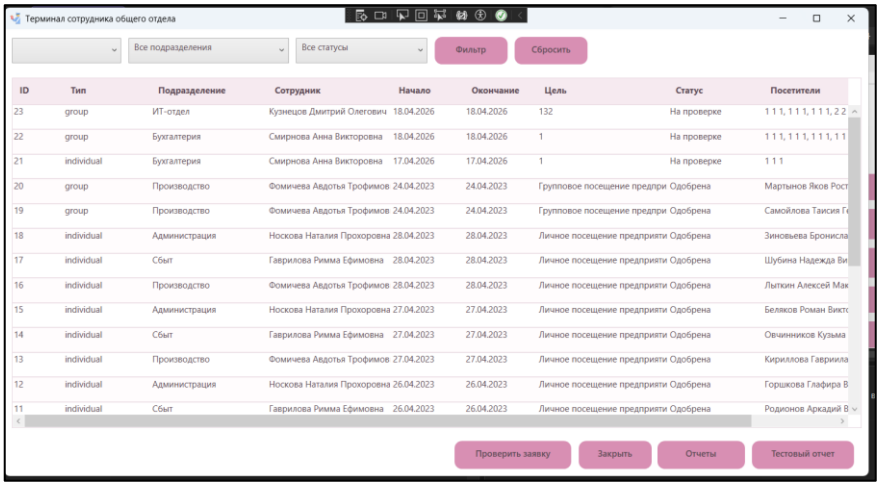

Рисунок 5 – Терминал общего отдела.

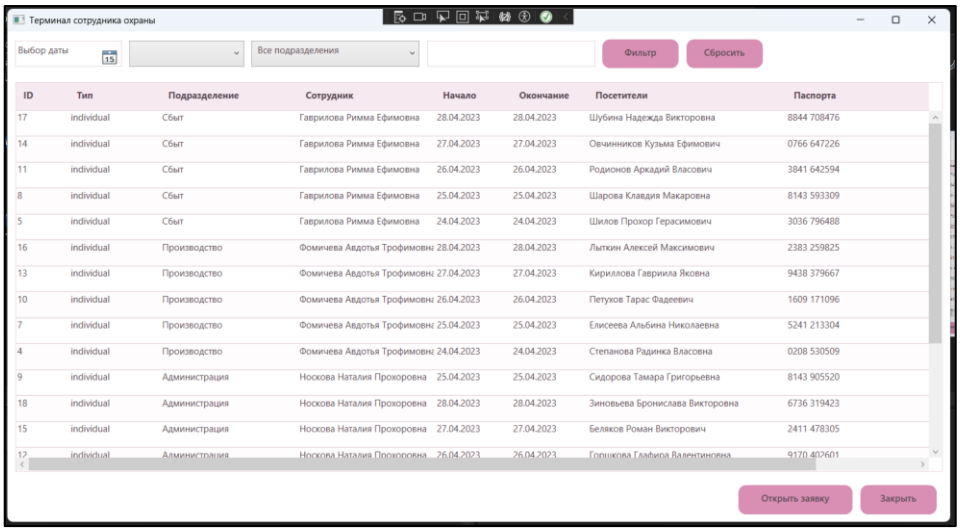

Рисунок 6 – Терминал охраны.

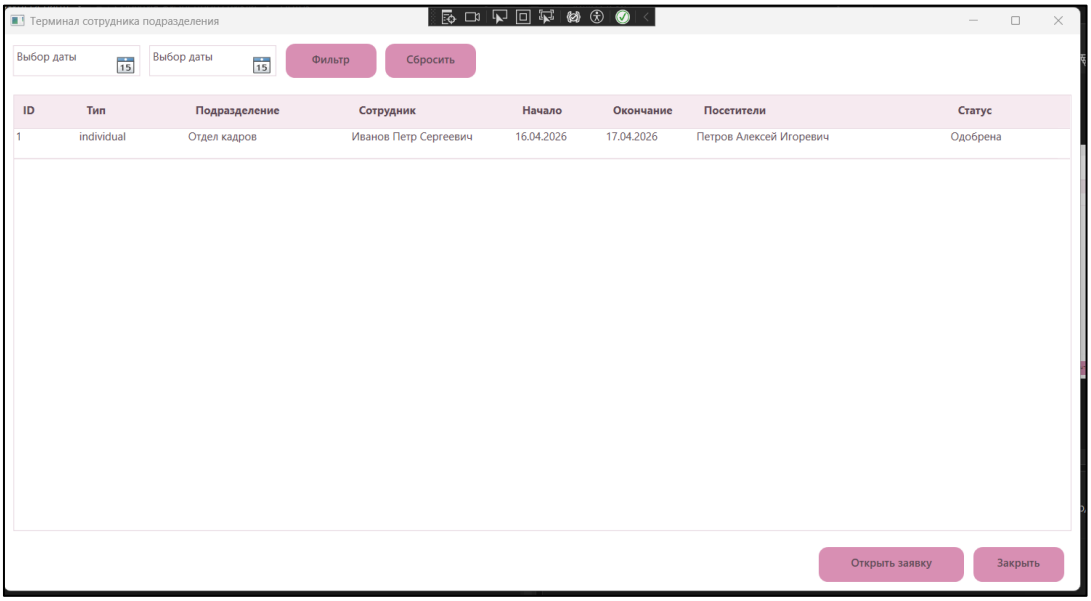

Рисунок 7 – Терминал подразделения.

5. Создание запросов и отчетов в соответствии с заданием (представления, хранимые процедуры, триггеры)

В базе данных и в программной части системы были реализованы запросы, представления, функции, триггеры и отчеты, обеспечивающие автоматизацию обработки информации. Эти механизмы используются для просмотра заявок, фильтрации данных, автоматического формирования логинов, контроля посещений и построения отчетов по посещаемости.

5.1. Представления

Для удобства получения сводной информации по заявкам было создано представление view_list_requests. Данное представление объединяет данные из нескольких таблиц: заявки, типы заявок, подразделения, сотрудники, статусы, посетители и связи заявок с посетителями.

Это позволяет одним запросом получить полную информацию по заявке без необходимости каждый раз вручную объединять таблицы.

Код представления

```
DROP VIEW IF EXISTS keeperpro.view_list_requests;

CREATE VIEW keeperpro.view_list_requests AS
SELECT
    a.application_id AS "ApplicationId",
    a.user_id AS "UserId",
    at.type_name AS "ApplicationType",
    d.department_name AS "DepartmentName",
    CONCAT(e.last_name, ' ', e.first_name, ' ', COALESCE(e.middle_name, '')) AS "EmployeeName",
    a.date_start AS "DateStart",
    a.date_end AS "DateEnd",
    a.visit_purpose AS "VisitPurpose",
    a.note AS "Note",
    s.status_name AS "StatusName",
    a.rejection_reason AS "RejectionReason",
    a.created_at AS "CreatedAt",
    a.updated_at AS "UpdatedAt",
    v.visitor_id AS "VisitorId",
    v.last_name AS "VisitorLastName",
    v.first_name AS "VisitorFirstName",
    v.middle_name AS "VisitorMiddleName",
    v.phone AS "VisitorPhone",
    v.email AS "VisitorEmail",
    v.organization AS "VisitorOrganization",
    v.birth_date AS "VisitorBirthDate",
    v.passport_series AS "PassportSeries",
    v.passport_number AS "PassportNumber",
    av.visitor_order AS "VisitorOrder"
FROM keeperpro.applications a
JOIN keeperpro.application_types at
    ON a.application_type_id = at.application_type_id
JOIN keeperpro.departments d
    ON a.department_id = d.department_id
JOIN keeperpro.employees e
    ON a.employee_id = e.employee_id
JOIN keeperpro.application_statuses s
    ON a.status_id = s.status_id
LEFT JOIN keeperpro.application_visitors av
    ON a.application_id = av.application_id
LEFT JOIN keeperpro.visitors v
    ON av.visitor_id = v.visitor_id;
```

Рисунок 8 – Представление.

Описание работы представления

Данное представление используется для:

- просмотра списка заявок;
- формирования отчетов;
- получения полной информации о заявке;
- вывода данных по посетителям, подразделениям и статусам в одном наборе данных.

Таким образом, представление упрощает разработку интерфейсов и запросов, так как не нужно повторно писать сложные JOIN.

5.2. Хранимые процедуры / функции

Для фильтрации заявок была реализована функция filtering_requests, которая позволяет отбирать данные по типу заявки, подразделению и статусу.

Функция возвращает таблицу, которую можно использовать в отчетах, в интерфейсе программы и при проверке данных.

Код функции фильтрации

```
DROP FUNCTION IF EXISTS keeperpro.filtering_requests(text, text, text);

CREATE OR REPLACE FUNCTION keeperpro.filtering_requests(
    p_application_type text DEFAULT NULL,
    p_department_name text DEFAULT NULL,
    p_status_name text DEFAULT NULL
)
RETURNS TABLE (
    "ApplicationId" int,
    "UserId" int,
    "ApplicationType" text,
    "DepartmentName" text,
    "EmployeeName" text,
    "DateStart" timestamp with time zone,
    "DateEnd" timestamp with time zone,
    "VisitPurpose" text,
    "Note" text,
    "StatusName" text,
    "RejectionReason" text,
    "CreatedAt" timestamp with time zone,
    "UpdatedAt" timestamp with time zone,
    "VisitorId" int,
    "VisitorLastName" text,
    "VisitorFirstName" text,
```

```
    "VisitorMiddleName" text,
    "VisitorPhone" text,
    "VisitorEmail" text,
    "VisitorOrganization" text,
    "VisitorBirthDate" timestamp with time zone,
    "PassportSeries" text,
    "PassportNumber" text,
    "VisitorOrder" int
)
LANGUAGE sql
AS $$
    SELECT
        v."ApplicationId",
        v."UserId",
        v."ApplicationType"::text,
        v."DepartmentName"::text,
        v."EmployeeName"::text,
        v."DateStart",
        v."DateEnd",
        v."VisitPurpose"::text,
        v."Note"::text,
        v."StatusName"::text,
        v."RejectionReason"::text,
        v."CreatedAt",
        v."UpdatedAt",
        v."VisitorId",
        v."VisitorLastName"::text,
        v."VisitorFirstName"::text,
        v."VisitorMiddleName"::text,
        v."VisitorPhone"::text,
        v."VisitorEmail"::text,
        v."VisitorOrganization"::text,
        v."VisitorBirthDate",
        v."PassportSeries"::text,
        v."PassportNumber"::text,
        v."VisitorOrder"
    FROM keeperpro.view_list_requests v
    WHERE
        (p_application_type IS NULL OR v."ApplicationType" = p_application_type)
        AND (p_department_name IS NULL OR v."DepartmentName" = p_department_name)
        AND (p_status_name IS NULL OR v."StatusName" = p_status_name)
    ORDER BY v."ApplicationId" DESC, v."VisitorOrder" ASC;
$$;
```

Рисунок 9 – Фильтрация.

Описание работы функции

Функция фильтрации позволяет:

- вывести все заявки без ограничений;
- показать только индивидуальные или только групповые заявки;
- отобрать заявки конкретного подразделения;
- отобрать заявки по статусу;
- комбинировать фильтры между собой.

Это обеспечивает удобство выборки и централизует логику отбора данных на стороне базы данных.

Примеры вызова функции

```
SELECT * FROM keeperpro.filtering_requests();
SELECT * FROM keeperpro.filtering_requests('group', NULL, NULL);
SELECT * FROM keeperpro.filtering_requests(NULL, 'ИТ-отдел', 'На проверке');
```

Рисунок 10 – Вызов функции.

Место под скриншот процедуры/функции фильтрации: [ВСТАВИТЬ СКРИНШОТ]

5.3. Триггеры

В системе была реализована автоматическая серверная логика формирования уникального логина для пользователя при регистрации. Для этого использовался триггер, который срабатывает после вставки новой записи в таблицу users.

Логин формируется по правилу:

- берется часть email до символа @;
- к ней добавляется идентификатор пользователя.

Например:

- ivan@mail.ru → ivan5
- testuser@mail.ru → testuser12

Добавление поля login

```
ALTER TABLE keeperpro.users
ADD COLUMN IF NOT EXISTS login varchar(100);
```

Рисунок 11 – Поле LOGIN.

Функция для триггера

```
CREATE OR REPLACE FUNCTION keeperpro.set_user_login()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
   email_prefix text;
BEGIN
   email_prefix := split_part(NEW.email, '@', 1);

   UPDATE keeperpro.users
   SET login = email_prefix || NEW.user_id
   WHERE user_id = NEW.user_id;

   RETURN NEW;
END;
$$;
```

Рисунок 12 – Функция для триггера.

Создание триггера

```
DROP TRIGGER IF EXISTS trg_users_set_login ON keeperpro.users;

CREATE TRIGGER trg_users_set_login
AFTER INSERT ON keeperpro.users
FOR EACH ROW
EXECUTE FUNCTION keeperpro.set_user_login();
```

Рисунок 13 – Создание триггера.

Описание работы триггера

После регистрации нового пользователя:

1. в таблицу users вставляется запись;
2. срабатывает триггер;
3. система берет email пользователя;
4. выделяет часть email до @;
5. добавляет user_id;
6. записывает итоговый логин в поле login.

Этот механизм автоматизирует формирование логинов и исключает их ручной ввод.

5.4. Отчеты

В терминале сотрудника общего отдела были реализованы отчеты, необходимые для контроля посещений и анализа данных.

5.4.1. Отчет по количеству посещений

Был реализован отчет, который показывает количество посещений:

- за день;
- за месяц;
- за год;

с возможностью группировки по подразделениям.

Для формирования отчета использовались данные из таблицы visit_logs, где хранится информация о фактических посещениях.

Код сервиса отчета по количеству посещений

```
using KeeperProWpf.Data;
using KeeperProWpf.Office.ViewModels;
using Microsoft.EntityFrameworkCore;

namespace KeeperProWpf.Services
{
    public class OfficeReportService
    {
        private DateTime ToUtcDate(DateTime dt)
        {
            return DateTime.SpecifyKind(dt.Date, DateTimeKind.Utc);
        }

        public async Task<List<VisitCountReportItem>> GetVisitCountReportAsync(string
periodType, DateTime selectedDate)
        {
            using var db = new AppDbContext();

            DateTime startUtc;
            DateTime endUtc;

            var baseDate = ToUtcDate(selectedDate);

            if (periodType == "День")
            {
                startUtc = baseDate;
                endUtc = startUtc.AddDays(1);
            }
            else if (periodType == "Месяц")
            {
```

```
            startUtc = new DateTime(baseDate.Year, baseDate.Month, 1, 0, 0, 0,
DateTimeKind.Utc);
            endUtc = startUtc.AddMonths(1);
        }
        else
        {
            startUtc = new DateTime(baseDate.Year, 1, 1, 0, 0, 0, DateTimeKind.Utc);
            endUtc = startUtc.AddYears(1);
        }

        var query = db.VisitLogs
            .Include(vl => vl.Application)
                .ThenInclude(a => a.Department)
            .Where(vl =>
                vl.EntryTime.HasValue &&
                vl.EntryTime.Value >= startUtc &&
                vl.EntryTime.Value < endUtc);

        return await query
            .GroupBy(vl => vl.Application != null && vl.Application.Department != null
                ? vl.Application.Department.DepartmentName
                : "Неизвестно")
            .Select(g => new VisitCountReportItem
            {
                DepartmentName = g.Key,
                VisitCount = g.Count()
            })
            .OrderBy(x => x.DepartmentName)
            .ToListAsync();
    }
  }
}
```

Рисунок 14 – Отчет по количеству посещений.

Описание работы отчета

Этот отчет:

- берет журнал посещений;
- выделяет записи за выбранный период;
- группирует данные по подразделениям;
- подсчитывает число посещений для каждого подразделения.

5.4.2. Список лиц, находящихся на территории

Также был реализован отчет, показывающий список лиц, которые находятся на территории организации в текущий момент.

Условием нахождения на территории считается:

- время входа указано;
- время общего выхода отсутствует;
- время выхода из подразделения отсутствует.

Код сервиса для списка лиц на территории

```
public async Task<List<CurrentPersonsReportItem>>
GetCurrentPersonsReportAsync()
{
   using var db = new AppDbContext();


   return await db.VisitLogs
      .Include(vl => vl.Application)
         .ThenInclude(a => a.Department)
      .Include(vl => vl.Visitor)
      .Where(vl =>
         vl.EntryTime.HasValue &&
         !vl.ExitTime.HasValue &&
         !vl.DepartmentExitTime.HasValue)
      .Select(vl => new CurrentPersonsReportItem
      {
         DepartmentName = vl.Application != null && vl.Application.Department
!= null
            ? vl.Application.Department.DepartmentName
            : "Неизвестно",
         VisitorFullName = vl.Visitor != null
            ? (vl.Visitor.LastName + " " + vl.Visitor.FirstName + " " +
(vl.Visitor.MiddleName ?? "")).Trim()
            : "",
         Passport = vl.Visitor != null
            ? (vl.Visitor.PassportSeries + " " + vl.Visitor.PassportNumber)
            : "",
         EntryTime = vl.EntryTime.HasValue
            ? vl.EntryTime.Value.ToString("dd.MM.yyyy HH:mm")
            : ""
      })
      .OrderBy(x => x.DepartmentName)
      .ThenBy(x => x.VisitorFullName)
      .ToListAsync();
}
```

Рисунок 15 – Лица на территории.

Описание работы отчета

Этот отчет:

- обращается к журналу посещений;
- выбирает посетителей, у которых зафиксирован вход;
- проверяет, что время выхода еще не указано;
- выводит ФИО, паспорт, подразделение и время входа.

5.4.3. Автоматическое формирование отчетов ТБ

В терминале общего отдела была реализована функция автоматического формирования отчетов ТБ по количеству посетителей каждые 3 часа. Отчеты сохраняются на ПК сотрудника в папке **«Отчеты ТБ»**.

При первом запуске создается папка:

- Документы\Отчеты ТБ

Ежедневно внутри нее создается папка:

- ДД_ММ_ГГГГ

Внутри сохраняются отчеты за каждые 3 часа.

Код сервиса автоматического отчета

```
using KeeperProWpf.Data;
using Microsoft.EntityFrameworkCore;
using System.Windows.Threading;

namespace KeeperProWpf.Services
{
    public class OfficeAutoReportService
    {
        private readonly DispatcherTimer _timer = new DispatcherTimer();

        public void Start()
        {
            EnsureBaseFolders();

            _timer.Interval = TimeSpan.FromMinutes(1);
            _timer.Tick += Timer_Tick;
            _timer.Start();
```

```
    }

    private async void Timer_Tick(object? sender, EventArgs e)
    {
        DateTime now = DateTime.Now;

        if (now.Minute == 0 && (now.Hour % 3 == 0))
        {
            string markerFile = GetMarkerFilePath(now);

            if (!File.Exists(markerFile))
            {
                await GenerateVisitorsCountReportAsync(now);
                File.WriteAllText(markerFile, now.ToString("dd.MM.yyyy
HH:mm"));
            }
        }
    }

    private void EnsureBaseFolders()
    {
        string reportsRoot = GetReportsRootFolder();

        if (!Directory.Exists(reportsRoot))
            Directory.CreateDirectory(reportsRoot);

        string todayFolder = GetTodayFolder();
        if (!Directory.Exists(todayFolder))
            Directory.CreateDirectory(todayFolder);
    }

    private string GetReportsRootFolder()
    {
        string documents =
Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments);
        return Path.Combine(documents, "Отчеты ТБ");
    }

    private string GetTodayFolder()
    {
        string reportsRoot = GetReportsRootFolder();
        string folderName = DateTime.Now.ToString("dd_MM_yyyy");
        return Path.Combine(reportsRoot, folderName);
    }
```

```
        private string GetMarkerFilePath(DateTime dt)
        {
            string todayFolder = GetTodayFolder();
            string markerName = $"report_{dt:HH_mm}.done";
            return Path.Combine(todayFolder, markerName);
        }

        public async Task GenerateVisitorsCountReportAsync(DateTime
reportTime)
        {
            using var db = new AppDbContext();

            DateTime endTime = reportTime.ToUniversalTime();
            DateTime startTime = endTime.AddHours(-3);

            var items = await db.VisitLogs
                .Include(vl => vl.Application)
                    .ThenInclude(a => a.Department)
                .Where(vl =>
                    vl.EntryTime.HasValue &&
                    vl.EntryTime.Value >= startTime &&
                    vl.EntryTime.Value < endTime)
                .GroupBy(vl => vl.Application != null &&
vl.Application.Department != null
                    ? vl.Application.Department.DepartmentName
                    : "Неизвестно")
                .Select(g => new
                {
                    DepartmentName = g.Key,
                    VisitCount = g.Count()
                })
                .OrderBy(x => x.DepartmentName)
                .ToListAsync();

            string todayFolder = GetTodayFolder();
            if (!Directory.Exists(todayFolder))
                Directory.CreateDirectory(todayFolder);

            string fileName =
$"Отчет_ТБ_{reportTime:dd_MM_yyyy_HH_mm}.txt";
            string fullPath = Path.Combine(todayFolder, fileName);

            using var writer = new StreamWriter(fullPath, false,
System.Text.Encoding.UTF8);
```

```
            await writer.WriteLineAsync("ОТЧЕТ ТБ");
            await writer.WriteLineAsync($"Период:
{startTime.ToLocalTime():dd.MM.yyyy HH:mm} -
{endTime.ToLocalTime():dd.MM.yyyy HH:mm}");
            await writer.WriteLineAsync("");

            if (items.Count == 0)
            {
                await writer.WriteLineAsync("За указанный период посещений не
найдено.");
            }
            else
            {
                await writer.WriteLineAsync("Подразделение | Количество
посетителей");
                await writer.WriteLineAsync("-------------------------------------");

                foreach (var item in items)
                {
                    await writer.WriteLineAsync($"{item.DepartmentName} |
{item.VisitCount}");
                }
            }
        }
    }
}
```

Рисунок 16 – Автоматический отчет.

Описание работы автоматического отчета

Данный сервис:

- автоматически запускается при работе терминала общего отдела;
- проверяет текущее время каждую минуту;
- если наступает время, кратное 3 часам, формирует отчет;
- сохраняет его на диск в папку **«Отчеты ТБ»**;
- создает служебный marker-файл, чтобы отчет не формировался повторно в тот же промежуток времени.

Вывод по разделу

Таким образом, в системе были реализованы:

- **представления** для сводного получения информации по заявкам;

- **функции** для фильтрации данных;
- **триггеры** для автоматического формирования логина;
- **отчеты** для анализа посещений и контроля присутствующих лиц;
- **автоматическое формирование отчетов ТБ** для целей безопасности.

Это позволило не только упростить выборку данных, но и автоматизировать важные процессы контроля доступа и отчетности.

6. Разработка модулей программы

В ходе практики была выполнена разработка настольных приложений системы **«ХранительПРО»**. Программа была разделена на несколько логических модулей, каждый из которых решает отдельную задачу.

6.1. Клиент посетителя

В клиенте посетителя были реализованы:

- регистрация и авторизация;
- оформление индивидуальной заявки;
- оформление групповой заявки;
- просмотр созданных заявок.

При создании групповой заявки был реализован ввод списка посетителей вручную без использования Excel.

6.2. Терминал сотрудника общего отдела

В терминале общего отдела были реализованы:

- вход по коду сотрудника;
- просмотр всех заявок;
- фильтрация заявок;
- проверка заявки;
- изменение статуса заявки;
- проверка по черному списку;
- составление отчетов;
- автоматическое формирование отчетов ТБ.

6.3. Терминал сотрудника охраны

В терминале сотрудника охраны были реализованы:

- вход по коду сотрудника;

- просмотр одобренных заявок;
- фильтрация по датам, типам и подразделениям;
- поиск по ФИО и номеру паспорта;
- разрешение доступа на территорию;
- фиксация времени входа и выхода.

6.4. Терминал сотрудника подразделения

В терминале сотрудника подразделения были реализованы:

- вход по коду сотрудника;
- просмотр заявок своего подразделения;
- подтверждение прихода и ухода посетителей;
- добавление посетителя в черный список с указанием причины;
- контроль временных ограничений на перемещение от проходной до подразделения.

При разработке модулей использовались:

- WPF;
- Entity Framework Core;
- PostgreSQL;
- XAML;
- C#.

Код программы был разбит на модели, сервисы, окна, сессии и вспомогательные классы. Это позволило сделать структуру проекта более понятной и удобной для поддержки.

7. Заключение

В ходе прохождения практики была разработана информационная система **«ХранительПРО»**, предназначенная для организации контроля физического доступа к значимому объекту.

В процессе работы были выполнены:

- анализ предметной области;
- построение UML2-диаграммы и ERD;
- выделение объектов и атрибутов;
- разработка словаря данных;

- проектирование и нормализация базы данных;
- реализация уровней доступа пользователей;
- создание запросов, представлений, процедур, триггеров и отчетов;
- разработка настольных приложений для посетителя, общего отдела, охраны и подразделения.

В результате была создана работоспособная система, которая позволяет автоматизировать оформление заявок, согласование посещений, контроль нахождения посетителей на территории организации и формирование отчетности.

8. Приложения

1. Ссылка на Git-репозиторий: **[КЛИК]**